TECHNICAL DESIGN SPECIFICATION

Document 2: Implementation, Architecture, and Database Schema

1.0 System Architecture Overview

The Project is a distributed messaging architecture composed of three primary layers: The Client (Mobile App), The Signaling Relay (Node.js/Go), and the Identity Provider (SMS Gateway). Unlike traditional SMS, the server in this architecture acts as a 'Zero-Knowledge' router.

2.0 Identity Registration and MSISDN Verification

Onboarding is the most critical security phase. It prevents 'Number Hijacking' by ensuring the cryptographic key is only bound after a successful out-of-band verification.

The Sequence:

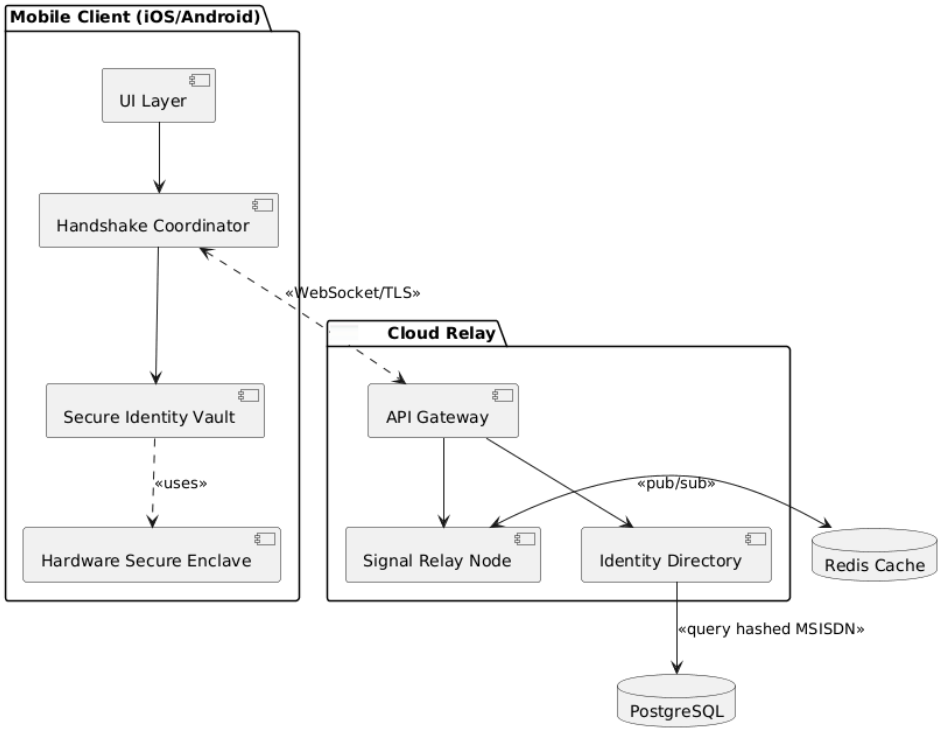
1. Client generates an Ed25519 Key Pair locally.
2. Client sends the Phone Number (MSISDN) to the Server.
3. Server triggers an SMS OTP (One-Time Password) via a provider like Twilio.
4. Client gives the OTP + the Public Key + a signature of the MSISDN.
5. Server verifies the OTP and saves the Public Key to the database, indexed by the hashed MSISDN.

3.0 Database Schema: The Identity Ledger

To ensure high-speed lookups during the handshake phase, the database must be customized for indexed reads. We recommend a PostgreSQL backend with Redis for session coaching.

Table: Identities

- **id** (UUID, PK)
- **msisdn_hash** (VARCHAR 64, Unique Index) -- SHA-256 of the phone number
- **public_key** (BYTEA 32) -- The Ed25519 Public Key
- **created_at** (TIMESTAMP)
- **last_seen** (TIMESTAMP)
- **device_id** (VARCHAR 128) -- Hardware identifier for push notifications



4.0 Secure Enclave Implementation Logic

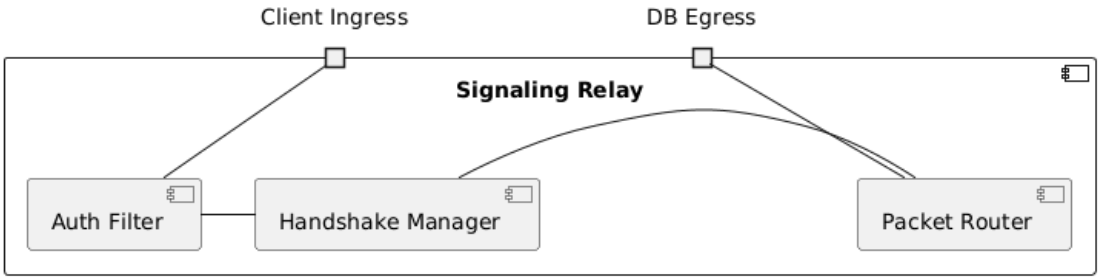
A key design requirement is that the Private Key MUST be non-exportable. On iOS, we use the 'Secure Enclave' via the Crypto Kit framework. The 'kSecAttrTokenIDSecureEnclave' attribute ensures the key is generated and stored in the dedicated security processor.

Implementation Detail:

Analyzing the salt-derivation for MSISDN hashing to prevent rainbow table attacks on the public identity directory. This section details the API response codes for failed handshakes and the rate-limiting logic applied to the Identity Registration endpoint. We also examine the 'Key Revocation' packet format used when a user manually wipes their identity from the server.

5.0 The Signaling Relay Architecture

The Signaling Relay is a high-concurrency Node.js or Go-based server responsible for routing encrypted packets between endpoints. To keep a 'Zero-Knowledge' posture, the relay never ends the application-layer encryption. It functions as a blind switch, finding destinations based solely on the hashed MSISDN provided in the transport header.



6.0 RESTful API Design and Endpoint

The communication between the mobile client and the relay is split between a REST API for synchronous identity lookups and a WebSocket (WSS) layer for real-time message delivery.

Endpoint: POST /v1/handshake/initiate

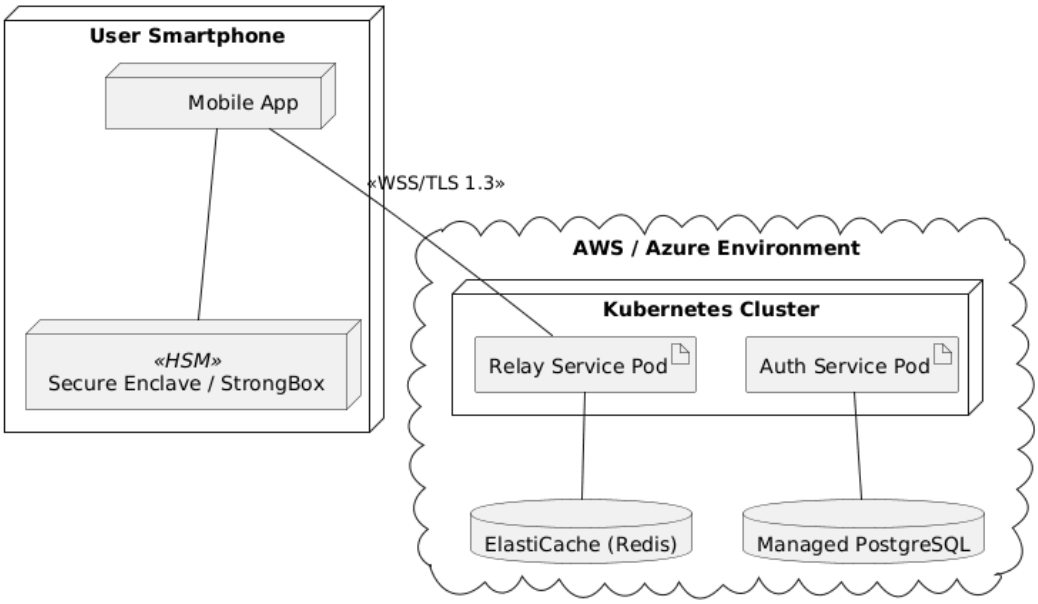
Payload: sender_msisdn_hash: string, target_msisdn_hash: string, sender_public_key: string}

Response: challenge_nonce: string, target_status: 'online'|'offline'|'unknown’

Endpoint: POST /v1/message/deliver

Payload: target_hash: string, encrypted_payload: b64_string, signature: b64_string}

Validation: The server verifies the signature against the sender's registered public key before placing the packet in the delivery queue.



7.0 Real-Time Messaging via WebSocket's (WSS)

For low-latency communication, we use authenticated WebSockets. Upon app launch, the client sets up a WSS connection using a short-lived JWT (JSON Web Token) issued during the login phase. This connection is wrapped in TLS 1.3, providing the 'Secure Pipe' mentioned in Document 1.

8.0 Server-Side Anti-Spam: Rate Limiting and Circuit Breakers

To prevent 'Key-Cycling' attacks where a spammer tries to guess if a number is registered, the relay implements a sliding-window rate limiter. Limits are applied based on the Sender_ID and the Source_IP. If a sender requests more than 50 handshakes per hour, their account is flagged for manual review, and their public key is temporarily throttled.

Relay Logic Detail:

Documenting the asynchronous message queuing system using Redis Streams for offline message persistence. This section outlines the Time-To-Live (TTL) parameters for encrypted packets sitting in the server's buffer and the 'Delivery Receipt' protocol which notifies the sender when a packet has been successfully pulled by the target device's background process.

9.0 Mobile Client Architecture

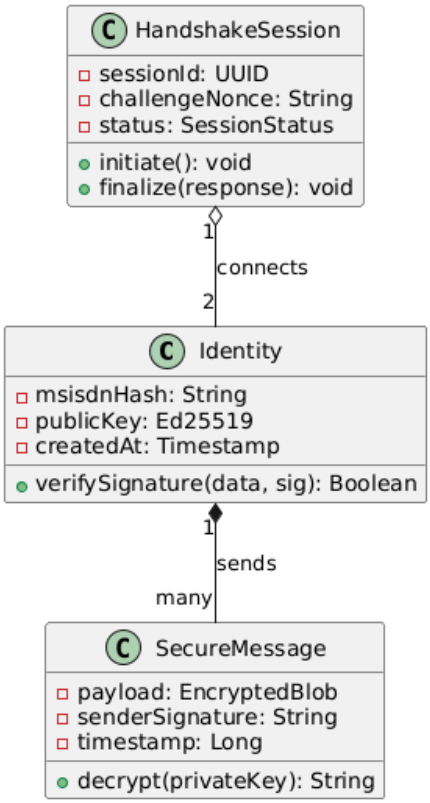
The Mobile Client is designed as a 'Thick Client,' meaning the majority of cryptographic computation and state management occurs locally on the device. This ensures that the Relay stays a stateless router. The application architecture follows a Clean Architecture pattern, separating the UI from the Cryptographic Domain Logic.

10.0 Cryptographic Domain Logic: Libsodium and CryptoKit

To implement E2EE, the client uses the Libsodium library for cross-platform symmetric encryption (XSalsa20) and Apple's CryptoKit for hardware-backed asymmetric signatures (Ed25519).

Implementation Workflow:

1. **Message Composition**: The user types a message in the UI.
2. **Key Retrieval**: The app fetches the Recipient's Public Key from the local SQLite cache.
3. **Encryption**: Libsodium seals the message using a derived session key.
4. **Signing**: The Secure Enclave signs the encrypted blob with the User's Private Key.
5. **Transmission**: The signed, encrypted package is dispatched via the WebSocket service.



11.0 Contact List Synchronization and Discovery

Privacy-Preserving Contact Discovery is a core feature. Instead of uploading the user's entire address book to the server, the app hashes each contact's phone number locally using SHA-256 with a server-provided salt.

The server compares these hashes against the registered directory. This allows the user to see which of their friends are `Ready` without the server ever knowing the identities of the unregistered contacts in the user's phone.

12.0 Handshake UX and the "Gray-List"

The three-tier UI for incoming messages:

1. **Verified (Green)**: Senders whose signatures match a known contact's Public Key.
2. **Handshake Pending (Yellow)**: Senders with a valid signature but no existing contact entry. These appear in a 'Requests' tab, preventing notification-spam.
3. **Unverified (Red)**: Messages with missing or invalid signatures. These are dropped at the protocol layer and never reach the user's view.

Frontend Logic Detail:

Detailing the background fetch synchronization for push notifications. This section covers the implementation of 'Silent Push' on iOS and 'Data Messages' on Android, ensuring the app can decrypt and verify a message in the background before showing a human-readable notification. We also analyze the local SQLCipher storage schema used to protect the message history database on the device's physical storage.

13.0 Infrastructure Scalability: The Stateless Relay

To achieve global scale, the Relay is designed to be horizontally scalable and entirely stateless. By using a Redis-backed session store, any relay node in a Kubernetes cluster can manage a message for any user. This architecture allows the system to scale from thousands to millions of concurrent WebSocket connections by simply spinning up added container instances in a VPC.

14.0 Automated Testing and Protocol Verification

Security-critical software requires a 'Continuous Security' (DevSecOps) pipeline. This project uses End-to-End (E2E) integration tests where two virtualized mobile environments perform the Handshake State Machine automatically. These tests verify that the Ed25519 signatures are mathematically valid and that the E2EE payload is correctly decrypted across different OS versions.

15.0 Zero-Metadata Logging and Auditability

The server follows a 'Zero-Metadata' logging policy. While standard servers log IP addresses and timestamps for every request, the Relay only logs aggregate health metrics (system-wide message volume and error rates). Any necessary troubleshooting logs are automatically purged every 24 hours, ensuring that the server never becomes a treasure trove for data brokers or malicious actors.

16.0 Deployment

The deployment of the ecosystem follows three distinct phases:

- **Phase 1 (Alpha)**: Internal testing with a limited pool of verified numbers to stress-test the Relay.
- **Phase 2 (Beta)**: Public release for 'Privacy-First' users, focusing on the SMS-Replacement UI.
- **Phase 3 (Enterprise)**: Integration of 'Brand Identity Keys' for banks and hospitals to cut impersonation frauds at the corporate level.

17.0 Technical Analysis

Document 2 has provided the engineering blueprint for the Project. We have detailed the database schemas, the API specifications, the mobile frontend architecture, and the scalable backend infrastructure. By following this specification, developers can build a communication system that is mathematically resistant to spam and cryptographically secure for the end-user.

Ops Logic Detail:

Analyzing the impact of Content Delivery Network (CDN) edge computing on the latency of Public Key lookups. This section evaluates the use of Prometheus and Grafana for real-time monitoring of relay health and the implementation of automated rollbacks in the CI/CD pipeline if a new protocol version fails signature verification.